EXPLORATORY DATA ANALYSIS

Chapter 6

Exploring Date and Time Variables

A practical and quantitative approach to understanding temporal data with Python and Pandas

Chapter focus: parsing dates, timestamps, time zones, calendar components, durations, intervals, temporal gaps, and time-based patterns.

Scope boundary: this chapter teaches you to inspect and understand temporal data. It does not cover time-series forecasting, feature engineering, resampling for modeling, or model training.

Learning Objectives

- Distinguish date-only values, times of day, timestamps, durations, and intervals.
- Recognize why dates imported as strings or mixed object columns require careful inspection.
- Parse temporal data safely with Pandas and diagnose parsing failures.
- Understand timestamp precision, time zones, UTC offsets, and daylight-saving complications.
- Extract and summarize calendar components without confusing them with causal explanations.
- Measure elapsed time, gaps, event spacing, and coverage.
- Explore temporal distributions and patterns using quantitative summaries and Matplotlib's object-oriented API.
- Recognize common temporal data-quality problems, including impossible dates, mixed formats, duplicate timestamps, and inconsistent time zones.

Review of Earlier Chapters

Chapter 1 — What Is Exploratory Data Analysis?

EDA is an iterative investigation of a dataset, not simply a collection of plots. For temporal variables, the same three questions remain central: What data do I have? Can I trust it? What patterns does it contain? A date pattern is a description first; an explanation requires context and further evidence.

Chapter 2 — Understanding the Dataset

Remember observational units, rows, columns, granularity, metadata, and the data-generating process. A timestamp may describe when an event occurred, when it was recorded, when a system processed it, or when a file was exported. Those meanings are not interchangeable.

Chapter 3 — Data Quality Assessment

Apply completeness, validity, consistency, uniqueness, accuracy, and timeliness to temporal columns. A non-null date can still be invalid; a valid timestamp can still be inaccurate; duplicate timestamps do not necessarily mean duplicate events.

Chapter 4 — Exploring Numerical Variables

Temporal analysis often creates numerical quantities such as elapsed seconds, days between events, or event counts per period. The Chapter 4 toolkit—center, spread, quantiles, distribution shape, and outlier investigation—helps describe these quantities.

Important: the previous chapter, Chapter 5, is reviewed separately below, as requested by the course structure.

Review of Chapter 5 — Exploring Categorical Variables

Categorical EDA used counts, proportions, frequency tables, cardinality, rare-category inspection, and cross-tabulation. These ideas remain useful when dates are grouped into weekdays, months, quarters, or other calendar categories.

- A weekday frequency table is a categorical summary derived from a temporal variable.
- Month names are categories, but calendar month numbers have a meaningful cyclic order.
- A count is only interpretable when its denominator and observation window are clear.
- Rare dates or categories should be investigated, not automatically removed.

Chapter 5 Assignment and Project — Discussion Guide

No submitted answers or notebook were included with this request, so this section gives the expected reasoning rather than pretending to grade specific answers.

- For category-frequency questions, report both counts and proportions when useful, and state the denominator.
- For high-cardinality variables, distinguish genuinely meaningful categories from identifiers or free-text labels.
- For rare categories, investigate whether they represent errors, unusual but valid cases, or a data-collection issue.
- The reusable categorical profiler should make category counts, proportions, missingness, cardinality, and rare levels reproducible.

What the previous coding project should have taught: how to build reusable summaries rather than repeatedly writing one-off inspection code; how to keep category labels consistent for analysis; and how to explain the meaning and limitations of a frequency table.

Preview of This Chapter

We now add a temporal dimension to the EDA toolkit. We will move from raw date strings to meaningful timestamps, then investigate calendar structure, elapsed time, time zones, event gaps, and temporal patterns.

1. The Different Kinds of Temporal Data

Type	Meaning	Example
Date	A calendar day, without a time of day	2026-11-03
Time of day	Clock time, often without a date	14:35:20
Timestamp / datetime	A point on a timeline	2026-11-03 14:35:20
Duration	An amount of elapsed time	2 days, 3 hours
Interval	A start and end, or a span between two points	09:00 to 17:00
Period	A calendar span such as month or quarter	2026-11

These types answer different questions. A date tells you which day; a timestamp identifies a point in time; a duration measures how long something took. A time of day such as 09:00 is not a complete timestamp because it does not specify a date or time zone.

2. Why Temporal Data Is Tricky

- Dates may be stored as strings, integers, or mixed object values.
- The same date can be written in several formats: 2026-11-03, 03/11/2026, or Nov 3, 2026.
- Day/month ambiguity can silently reverse meaning: 04/05/2026 may mean April 5 or May 4.
- A timestamp may be local time or UTC; without a time-zone convention, its meaning can be ambiguous.
- Some systems record event time while others record ingestion or processing time.
- Events may arrive late, be duplicated, or appear out of order.
- Calendar periods have unequal lengths; months are not all 30 days.

> **Rule:** never infer a date format or time-zone meaning from appearance alone when the data dictionary, source system, or collection process can establish it.

3. Inspect Before Parsing

Start with the raw values. Check data types, representative values, missingness, and the range of strings. Do not immediately overwrite the original column: retaining the raw value makes parsing failures auditable.

```
import pandas as pd
import numpy as np

df = pd.read_csv("events.csv")

print(df.shape)
print(df.dtypes)
print(df["event_time"].head(10))
print(df["event_time"].tail(10))
print(df["event_time"].isna().sum())
print(df["event_time"].nunique(dropna=True))
```

Ask: Is the column consistently formatted? Does it contain date only or date plus time? Are there blanks, placeholder strings, or multiple formats? Is the source's time zone documented?

4. Parsing Dates and Timestamps

Use `pd.to_datetime()` to convert recognized date/time values into Pandas datetime types. Use an explicit format when the format is known and consistent. For messy data, coercing failures to `NaT` can help reveal invalid values, but it must be followed by an audit.

```
raw = df["event_time"]

df["event_time_parsed"] = pd.to_datetime(
    raw,
    format="%Y-%m-%d %H:%M:%S",
    errors="coerce"
)

failed = raw.notna() & df["event_time_parsed"].isna()
print("Non-null values that failed parsing:", failed.sum())
print(df.loc[failed, "event_time"].value_counts().head(20))
```

The format string is a contract about the input. Common directives include:

Directive	Meaning	Example
%Y	Four-digit year	2026
%y	Two-digit year	26
%m	Month number	11
%d	Day of month	03
%H	Hour, 24-hour clock	14
%I	Hour, 12-hour clock	02
%M	Minute	35
%S	Second	20
%p	AM / PM marker	PM

For example, `03/11/2026` is ambiguous without a known convention. A parsing option cannot replace knowledge of the source format. When a dataset genuinely contains multiple formats, identify and handle each documented format explicitly instead of hoping a permissive parser guesses correctly.

5. Missing and Invalid Temporal Values

Pandas uses `NaT` (Not a Time) for missing or unparseable datetime values. Distinguish values that were originally missing from values that became NaT because parsing failed.

```
raw = df["event_time"]
parsed = pd.to_datetime(raw, errors="coerce")

originally_missing = raw.isna()
parse_failed = raw.notna() & parsed.isna()

print("Originally missing:", originally_missing.sum())
print("Failed to parse:", parse_failed.sum())
```

An invalid date such as February 30 is not made valid by conversion. It should be counted, inspected, and traced to its source. Avoid silently dropping invalid records during EDA.

6. Calendar Components

After parsing, the Pandas `.dt` accessor exposes calendar components. These are useful for describing the dataset's temporal composition.

```
dt = df["event_time_parsed"]

df["year"] = dt.dt.year
df["month"] = dt.dt.month
df["day"] = dt.dt.day
df["day_of_week"] = dt.dt.dayofweek
df["day_name"] = dt.dt.day_name()
df["hour"] = dt.dt.hour
df["quarter"] = dt.dt.quarter
df["is_month_end"] = dt.dt.is_month_end
```

Pandas numbers weekdays from Monday = 0 through Sunday = 6. Confirm the convention before interpreting a weekday chart. Calendar components are derived descriptions, not automatically meaningful predictors or explanations.

Useful calendar questions

- Which years and months are represented?
- Are all weekdays represented, and how many observations occur on each?
- Are records concentrated during business hours or around a particular hour?
- Are there missing calendar periods?
- Does the dataset cover complete months, or only partial months at its boundaries?

7. Time Zones and UTC

A timestamp without time-zone information is called timezone-naive. A timestamp that includes time-zone information is timezone-aware. UTC is a common reference standard for storing and comparing instants across locations.

Two timestamps that display different clock times may represent the same instant. For example, 12:00 UTC and 18:00 in Bangladesh (UTC+06:00) refer to the same instant.

```
# If the source values are known to be UTC:
utc_time = pd.to_datetime(df["event_time"], utc=True)

# Convert an aware UTC series to Bangladesh local time:
bd_time = utc_time.dt.tz_convert("Asia/Dhaka")
```

Critical distinction: `tz_localize()` assigns a time zone to a naive clock reading; `tz_convert()` converts an already-aware instant to another zone. Do not use localization as if it were conversion.

```
# Example: naive values are known to represent Dhaka local clock time
naive = pd.to_datetime(df["local_time"], errors="coerce")

dhaka_aware = naive.dt.tz_localize(
    "Asia/Dhaka",
    ambiguous="NaT",
    nonexistent="NaT"
)
```

Some regions observe daylight saving time. Certain local clock times can occur twice or not at all during clock changes. Bangladesh currently uses UTC+06:00 without seasonal daylight-saving changes, but datasets from other regions may require explicit handling. Preserve the original source time and document the assumptions used.

8. Durations, Differences, and Time Intervals

Subtracting two compatible timestamps produces a timedelta. This allows you to describe elapsed time between events, order delays, session lengths, and time between customer interactions.

```
df["start_time"] = pd.to_datetime(df["start_time"], errors="coerce")
df["end_time"] = pd.to_datetime(df["end_time"], errors="coerce")
```

```
df["elapsed"] = df["end_time"] - df["start_time"]
df["elapsed_hours"] = df["elapsed"].dt.total_seconds() / 3600

print(df["elapsed"].describe())
print(df["elapsed_hours"].quantile([0, .25, .5, .75, 1]))
```

Check for negative durations. They may indicate reversed columns, clock errors, time-zone mismatches, or a legitimate process in which the recorded end precedes the start. Do not assume which explanation is correct.

```
negative = df["elapsed"] < pd.Timedelta(0)
zero = df["elapsed"] == pd.Timedelta(0)

print("Negative intervals:", negative.sum())
print("Zero-length intervals:", zero.sum())
print(df.loc[negative, ["start_time", "end_time", "elapsed"]].head())
```

Use total seconds, minutes, or hours for elapsed-time calculations. Avoid treating a calendar month as a fixed number of days; calendar offsets and elapsed durations are different concepts.

9. Temporal Gaps and Event Spacing

For event logs, the difference between consecutive timestamps can reveal cadence, inactivity, bursts, and missing periods. First sort by the relevant entity and time. Otherwise, a difference may compare unrelated or out-of-order records.

```
events = df.sort_values(["customer_id", "event_time_parsed"]).copy()

events["gap"] = (
    events.groupby("customer_id")["event_time_parsed"].diff()
)
events["gap_hours"] = events["gap"].dt.total_seconds() / 3600

print(events["gap_hours"].describe())
```

Interpret the first gap in each group carefully: it is missing because there is no previous event in that group. Investigate unusually large gaps with the source's expected event frequency and observation window in mind.

10. Temporal Coverage and Completeness

A dataset may contain many records but still have poor temporal coverage. Inspect the earliest and latest timestamps, the number of distinct dates, and the distribution of records across calendar periods.

```
valid_time = df["event_time_parsed"].dropna()

print("First timestamp:", valid_time.min())
print("Last timestamp:", valid_time.max())
print("Distinct calendar dates:", valid_time.dt.date.nunique())

daily_counts = (
    df.dropna(subset=["event_time_parsed"])
      .set_index("event_time_parsed")
      .resample("D")
      .size()
)

print(daily_counts.head())
print(daily_counts.tail())
print("Days with zero recorded events:", (daily_counts == 0).sum())
```

A zero-count day can mean no events occurred, the source failed to collect data, or the day lies inside an incomplete observation window. EDA alone may not distinguish these. Compare against operational calendars, system logs, and collection documentation.

11. Quantitative Summaries for Temporal Variables

Temporal variables can be summarized in several complementary ways:

- Coverage: minimum and maximum timestamp, span, distinct days, and number of observed periods.
- Frequency: counts by day, week, month, weekday, or hour.
- Timing: distribution of hour-of-day and day-of-week.
- Spacing: median, quantiles, and extremes of inter-event gaps.

- Duration: center, spread, and unusual values of elapsed intervals.
- Completeness: missing timestamps, parsing failures, and empty periods.

For event counts, state the unit and period. '120 events' is incomplete; '120 events per observed calendar day across 30 days' is interpretable. If the observation window is partial, disclose it.

12. Visual EDA for Temporal Data

Visualizations should answer a defined question. Use Matplotlib's object-oriented interface: create a Figure and Axes with `fig, ax = plt.subplots()`, and call methods on the Axes.

A. Event counts over time

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(daily_counts.index, daily_counts.values)
ax.set_title("Recorded Events per Day")
ax.set_xlabel("Date")
ax.set_ylabel("Event count")
ax.grid(True, alpha=0.3)
fig.autofmt_xdate()
fig.tight_layout()
```

A line chart is useful when the x-axis is ordered time and the question concerns changes over time. It can expose bursts, gaps, gradual changes, and unusual periods. It does not explain why those changes occurred.

B. Distribution of hour of day

```
hours = df["event_time_parsed"].dt.hour.dropna()

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(hours, bins=np.arange(-0.5, 24.5, 1), edgecolor="black")
ax.set_xticks(range(24))
ax.set_xlabel("Hour of day")
ax.set_ylabel("Number of records")
ax.set_title("Distribution of Events by Hour")
fig.tight_layout()
```

This chart summarizes clock-hour concentration. It does not preserve the date sequence and therefore is not a trend chart.

C. Weekday counts

```
weekday_order = [
    "Monday", "Tuesday", "Wednesday", "Thursday",
    "Friday", "Saturday", "Sunday"
]
weekday_counts = (
    df["event_time_parsed"].dt.day_name()
      .value_counts()
      .reindex(weekday_order, fill_value=0)
)

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(weekday_counts.index, weekday_counts.values)
ax.set_title("Events by Day of Week")
ax.set_xlabel("Weekday")
ax.set_ylabel("Number of events")
ax.tick_params(axis="x", rotation=30)
fig.tight_layout()
```

This is a categorical frequency chart derived from dates. It should be interpreted alongside the observation window and any weekly operating schedule.

D. Distribution of inter-event gaps

```
gap_values = events["gap_hours"].dropna()

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(gap_values, bins=40, edgecolor="black")
ax.set_title("Distribution of Inter-event Gaps")
ax.set_xlabel("Gap (hours)")
ax.set_ylabel("Frequency")
fig.tight_layout()
```

Inter-event gaps are often right-skewed. If a few very large gaps dominate the scale, investigate them and consider a separate view or quantile summary rather than hiding them.

13. Common Temporal EDA Mistakes

- Parsing ambiguous date strings without confirming day/month order.
- Treating timezone-naive timestamps as UTC without evidence.
- Using tz_convert on naive timestamps or tz_localize on already-aware timestamps.
- Dropping parsing failures without reporting how many values were affected.
- Assuming a zero-event day means no activity rather than possible missing collection.
- Calculating event gaps before sorting within each entity.
- Comparing timestamps from different time zones without normalizing their instants.
- Treating month lengths as equal or using fixed-day assumptions for calendar periods.
- Interpreting weekday or hourly patterns without considering business schedules and sampling bias.
- Calling a temporal association a cause.

14. A Reusable Temporal Audit Function

The following starter function produces a compact audit of a date/time column. It intentionally reports parsing failures and coverage rather than silently cleaning the data.

```
def audit_datetime(series, fmt=None, utc=False):
    parsed = pd.to_datetime(
        series,
        format=fmt,
        errors="coerce",
        utc=utc
    )

    originally_missing = series.isna()
    parse_failed = series.notna() & parsed.isna()
    valid = parsed.dropna()

    result = {
        "rows": len(series),
        "originally_missing": int(originally_missing.sum()),
        "parse_failed": int(parse_failed.sum()),
        "valid_parsed": int(parsed.notna().sum()),
        "distinct_timestamps": int(valid.nunique()),
        "minimum": valid.min() if len(valid) else pd.NaT,
        "maximum": valid.max() if len(valid) else pd.NaT,
    }

    return result, parsed, series.loc[parse_failed]

audit, parsed, failures = audit_datetime(
    df["event_time"],
    fmt="%Y-%m-%d %H:%M:%S"
)

print(audit)
print(failures.value_counts().head(10))
```

A production-ready audit may also record the assumed source time zone, time-zone awareness, timestamp precision, expected collection cadence, and domain-specific valid ranges. The function is a reusable starting point, not a universal validator.

15. A Practical Temporal EDA Workflow

Step	Action	Output
1	Establish meaning and source conventions	Definition of the timestamp and its time zone
2	Inspect raw values and dtype	Initial format and quality assessment

Step	Action	Output
3	Parse while preserving raw values	Parsed datetime column and failure list
4	Audit missingness and invalid values	Counts and examples of failures
5	Check time-zone assumptions	Comparable, documented timestamps
6	Summarize coverage and calendar components	Temporal range and frequency tables
7	Calculate durations and event gaps	Elapsed-time distributions
8	Visualize relevant patterns	Plots matched to specific questions
9	Interpret with context	Findings, caveats, and follow-up questions

Chapter Summary

- Temporal data includes dates, times, timestamps, durations, intervals, and calendar periods.
- Inspect raw values and source conventions before parsing.
- Keep original values, count parsing failures, and investigate NaT values.
- Understand timezone-naive versus timezone-aware timestamps; localization and conversion are different operations.
- Use calendar components to summarize timing, while remembering that derived categories do not explain causes.
- Use timedeltas to examine durations and event gaps; sort records correctly before calculating consecutive gaps.
- Assess coverage and zero-count periods before claiming that no events occurred.
- Combine quantitative summaries and carefully chosen plots to describe temporal structure.

Assignment

Part A — Conceptual Questions

- Explain the difference between a date, a timestamp, a duration, and a time interval. Give one practical example of each.
- Why should you preserve the raw date/time column after parsing?
- What is the difference between a parsing failure and an originally missing value?
- Explain timezone-naive and timezone-aware timestamps.
- What is the difference between tz_localize() and tz_convert()?
- Why should event records be sorted by entity and timestamp before calculating inter-event gaps?
- Why does a day with zero recorded events not necessarily mean that no events occurred?
- Give two reasons why a weekday pattern should not automatically be interpreted as causal.

Part B — Quantitative Analysis

Consider these event timestamps, all documented as Dhaka local time:

```
2026-10-01 09:00:00
2026-10-01 09:15:00
2026-10-01 10:00:00
2026-10-02 09:30:00
2026-10-02 12:30:00
2026-10-04 08:00:00
```

- How many events are recorded on each date?

- Calculate the consecutive gaps in minutes after sorting.
- Find the minimum, median, and maximum inter-event gap.
- Identify the largest gap and propose at least two explanations that should be investigated before drawing a conclusion.
- What additional information would you need to decide whether the gap indicates missing data?

Part C — Data Quality Investigation

A CSV column contains the following raw values:

```
2026-05-04 08:30:00
04/05/2026 09:00
2026-05-04T10:15:00Z
not recorded
2026-02-30 11:00:00
2026-05-05 08:45:00
```

- Identify the ambiguities and invalid values.
- Explain why a single guessed format may be unsafe.
- Describe what source documentation you would request.
- Design a parsing audit that separates originally missing values from parsing failures.
- Explain how you would preserve the original values for traceability.

Coding Project — Temporal Data Profiler

Extend the cumulative EDA master notebook with a reusable temporal-analysis section. Do not create a disconnected one-off notebook: this project should become the temporal module of the evolving EDA toolkit.

Project requirements

- Create or load an event dataset containing an entity identifier and at least one date/time column. Include a start/end pair if available.
- Preserve the raw temporal column and create a parsed datetime column.
- Report original missing values, parsing failures, valid parsed values, distinct timestamps, minimum, and maximum.
- Document the expected input format and source time zone.
- Generate year, month, weekday, and hour summaries where those components are applicable.
- Calculate elapsed durations for start/end pairs and investigate negative or zero durations.
- For repeated events per entity, sort correctly and calculate inter-event gaps.
- Summarize temporal coverage and identify zero-count days or periods within the observed range.
- Create at least three Matplotlib visualizations using the object-oriented API: event counts over time, a calendar-component distribution, and a duration or gap distribution.
- Write at least five findings and at least three limitations or follow-up questions. Do not present descriptive patterns as causal conclusions.

Expected reusable functions: a datetime audit function, a calendar-frequency summary, and an inter-event-gap calculator. Add clear docstrings and return values so the functions can be reused on future client datasets.

Suggested deliverables: an updated cumulative EDA master notebook, a short written findings section, and exported plots. Keep the raw data and assumptions traceable.

Preview of Next Chapter

Chapter 7 — Descriptive Statistics: Beyond the Basics

We'll go beyond the familiar mean, median, variance, and standard deviation. We'll study robust statistics, trimmed and weighted means, measures of relative variability, quantile-based summaries, and how to choose statistics that accurately describe a dataset.